جامعة السلطان مولاي سليمان
Université Sultan Moulay Slimane
كلية العلوم التطبيقية - بني ملال
Faculté des Sciences Appliquées - Béni Mellal

OCP

PROJET DE FIN D'ÉTUDES

AI-Enhanced SOAR for Automated Incident Response in Enterprise SIEM

Une couche de classification par apprentissage automatique entre Wazuh et Shuffle

PRÉSENTÉ PAR

Hamza Saidi
Master Data Science et Sécurité des Systèmes d'Information

ENCADRANT PÉDAGOGIQUE

Pr. Mohamed El Fezazi
FSA Béni Mellal

ENCADRANTS PROFESSIONNELS

M. Fahmi Abderrahim
M. Erraji Hamza

ANNÉE UNIVERSITAIRE

2025 – 2026

DÉROULÉ

Plan de la présentation

01
Contexte et problématique
L'OCP, le problème des 100 alertes par seconde, les objectifs du projet

02
État de l'art et conception
Choix SIEM et SOAR, apprentissage automatique pour la détection, architecture en 5 couches

03
Réalisation
Dataset, 11 features, labellisation comportementale, Random Forest, API Flask, workflow Shuffle

04
Résultats et discussion
Performances, escalade comportementale, tests de bout en bout, limites et perspectives

Le cadre du projet : le Groupe OCP

Groupe Premier exportateur mondial de phosphate. Fondé en 1920, présent dans plus de 160 pays.

Site Benguerir, laverie de Gantour, département Maintenance. Stage de 4 mois, de mars à juillet 2026, en cybersécurité industrielle.

Enjeu Le programme OCP 4.0 relie les capteurs, l'automatisation et les données. Les systèmes industriels, longtemps isolés, deviennent accessibles depuis le réseau informatique : la surface d'attaque s'élargit.

Besoin Détecter les incidents et y répondre vite, malgré un volume d'alertes qui augmente.

1920
création du Groupe OCP

160+
pays où le groupe est présent

4 mois
de stage à Benguerir (mars à juillet 2026)

01 · PROBLÉMATIQUE

Le problème : un SOAR sans triage ne sert à rien

≈100

alertes par seconde générées par le SIEM de l'OCP en production

SANS TRIAGE

SIEM Wazuh → flux brut **100 alertes/s** → **SOAR** Shuffle → **Analystes saturés**

AVEC NOTRE COUCHE DE TRIAGE

SIEM Wazuh → **Classification ML** LOW · MEDIUM · HIGH → **SOAR** → **Seul l'utile arrive**

Sans filtre, le SOAR reçoit le même flux brut et reproduit **la saturation qu'il devait résoudre**. Les analystes retrouvent le même volume non priorisé, dans un autre outil.

Objectifs et exigences du système

CE QUE LE SYSTÈME DOIT FAIRE

BF1 Surveiller les logs en temps réel et générer des alertes

BF2 Transmettre chaque alerte au SOAR par webhook

BF3 Classer en LOW, MEDIUM ou HIGH avec un score de confiance

BF4 Escalader selon le comportement dans le temps

BF5 Bloquer l'IP attaquante pour les alertes HIGH

BF6 Notifier par Telegram et par e-mail

BF7 Exposer `/health` et `/stats`

CONTRAINTES À RESPECTER

< 2 s
temps de réponse de l'API

10 Go
de RAM et 4 vCPU pour toute l'infrastructure

100 %
des composants serveur sous Docker

Dataset
reproductible, généré depuis Wazuh

Choix de la pile SIEM et SOAR

Wazuh 4.9.2

SIEM

- Open source et gratuit
- Nombreuses règles **MITRE ATT&CK**
- Webhook natif vers le SOAR
- Image Docker officielle

Écartés : Splunk et IBM QRadar (commerciaux, coûteux), Elastic (webhook via Logstash)

Shuffle 2.2.0

SOAR

- Open source et gratuit
- Léger : tient dans 10 Go de RAM
- Blocs HTTP natifs
- Limite : pas de conditionnel natif, contournée par des conditions sur les flèches

Écartés : TheHive (webhook par plugin), Cortex XSOAR et Swimlane (commerciaux)

Ce que dit la littérature sur le ML appliqué aux IDS

Gu et al. (2019)

Random Forest bat SVM et Naive Bayes sur CICIDS2017 : **F1 de 98,7 %** sur la classe DoS.

Sharafaldin et al. (2018)

Les modèles entraînés sur des features réseau se généralisent mal d'un environnement à l'autre.

Catillo et al. (2022)

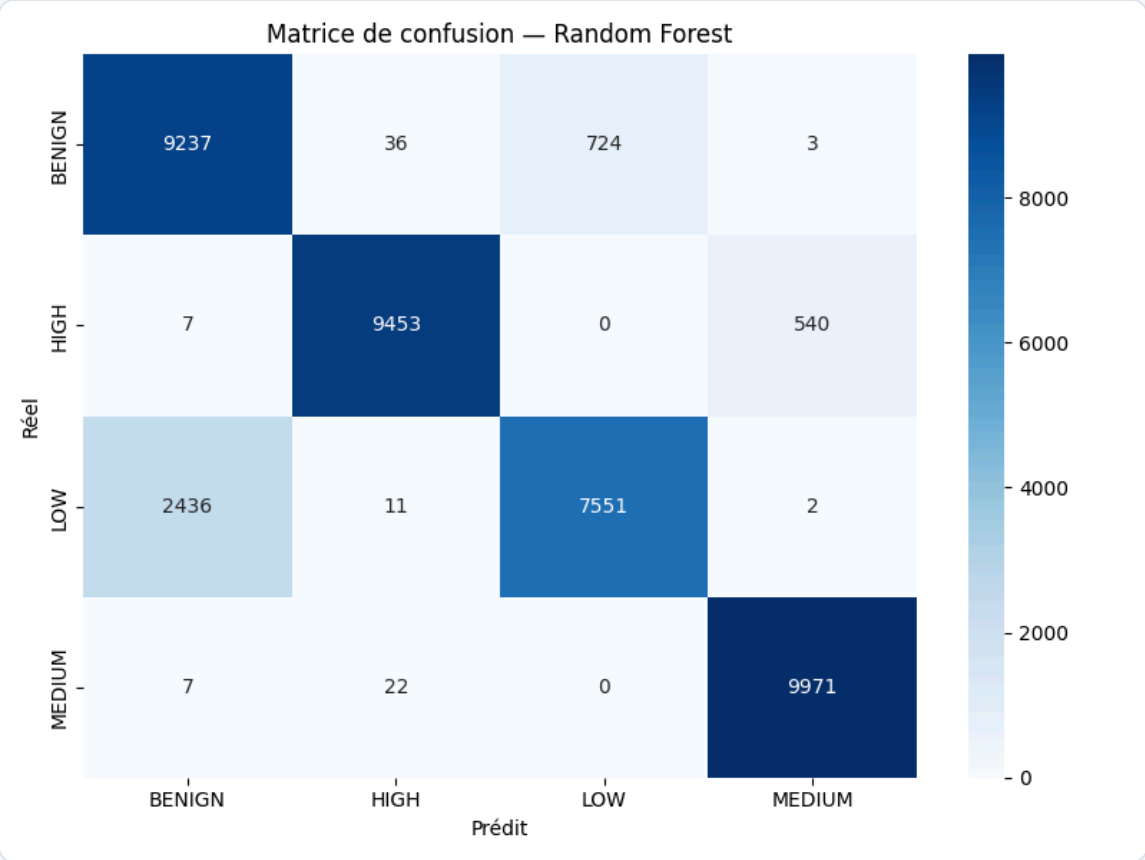
Le même problème de **domain shift** apparaît quand on applique ces modèles à des logs système.

Hindy et al. (2020)

Les features sémantiques (type d'événement, source, fréquence) sont plus stables que les features réseau.

Notre position : entraîner le modèle directement sur les alertes du SIEM cible (approche *domain-native*) et ajouter ce qui manque à la littérature : **la fréquence des alertes et la réputation de l'IP source.**

Pourquoi CICIDS2017 a été abandonné



Matrice de confusion, Random Forest sur CICIDS2017

60,9 %

du pouvoir discriminant vient de 5 features de trafic réseau (taille de paquets, port, débit). **Aucune n'existe dans une alerte Wazuh.**

2 436

exemples LOW prédits BENIGN par le Random Forest (2 410 pour XGBoost)

Incompatibilité structurelle : un mapping heuristique ne la corrige pas. D'où un **dataset natif Wazuh**.

Architecture en cinq couches

COUCHE 1

Simulation

simulate_attacks.py

4 types d'attaques : brute force SSH, scan de ports, XSS, connexions suspectes

→

COUCHE 2

SIEM

Wazuh 4.9.2

Détecte, corrèle, envoie l'alerte en JSON par webhook

→

COUCHE 3

SOAR

Shuffle 2.2.0

Reçoit l'alerte et appelle l'API de classification

→

COUCHE 4

IA

Flask + Random Forest

Calcule 11 features, renvoie un label et un score de confiance

→

COUCHE 5

Réponse

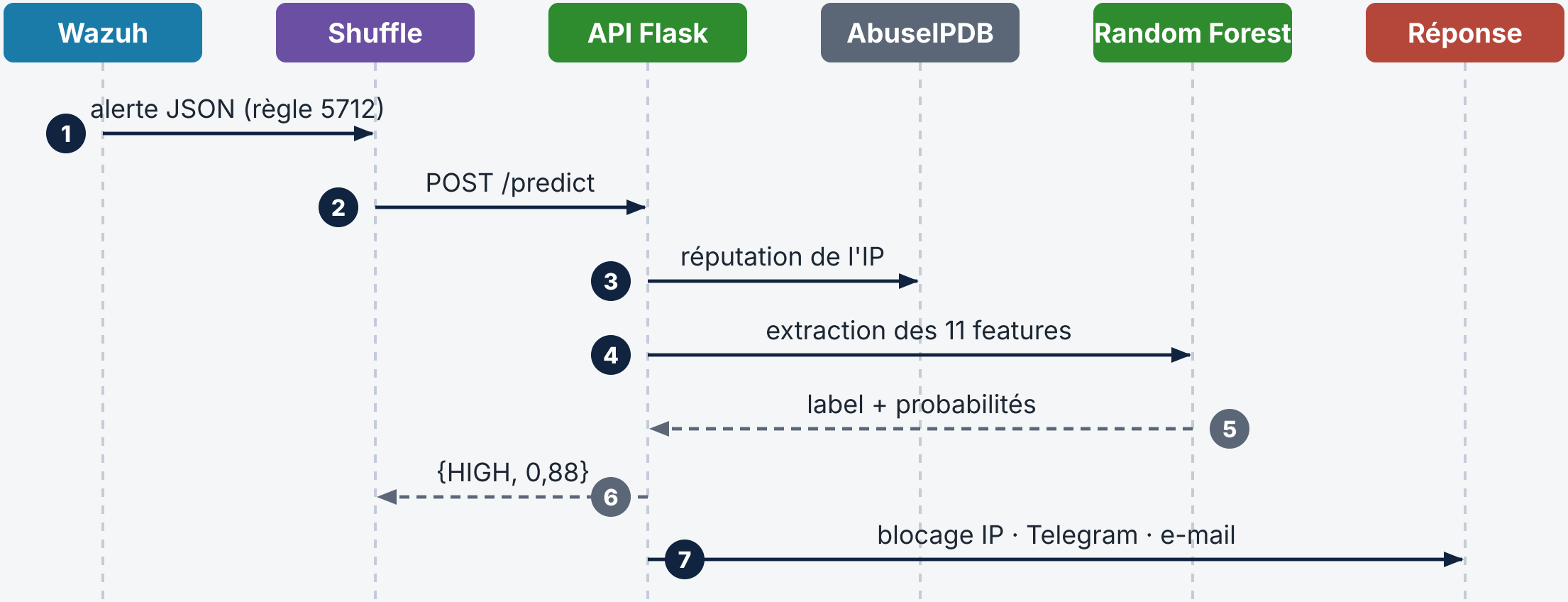
Actions automatiques

Blocage IP, Telegram, e-mail ou simple journal selon le label

```
// réponse de POST /predict
{ "label": "HIGH",
  "confidence": 0.88,
  "action": "BLOCK_AND_ALERT",
  "rule_id": "5712" }
```

Les couches ne se parlent qu'en **JSON** : on peut remplacer le modèle ou la plateforme SOAR sans toucher au reste.

Le parcours d'une alerte HIGH, étape par étape



Cas simulé : brute force SSH, règle Wazuh 5712 — **1,231 s** mesuré, objectif < 2 s

Le dataset : 3 945 alertes natives de Wazuh

CONSTRUIT EN TROIS COUCHES

1. Alertes organiques

Générées par `simulate_attacks.py` et détectées par Wazuh

2. Alertes injectées

Niveaux 10 et 12, difficiles à déclencher naturellement

3. IP publiques malveillantes

Pour entraîner la feature de réputation AbuseIPDB

DISTRIBUTION DES LABELS



Sans l'injection ciblée, HIGH n'aurait représenté que 5 % du dataset. Découpage 80 / 20 stratifié : **789 alertes de test**.

Les 5 342 alertes du dashboard Wazuh incluent des événements système (démarrages, SCA, rootcheck), exclus volontairement.

Les 11 features du modèle

Natives Wazuh (6)

Présentes dans chaque alerte

`rule_id_enc`	
`rule_level`	**26,85 %**
`groups_count`	
`has_mitre`	
`src_ip_private`	1,13 %
`fired_times`	5,17 %

Comportementales (4)

Calculées en temps réel

`alert_frequency_5min`	**13,10 %**
`repeat_rule_10min`	**13,84 %**
`hour_of_day`	
`is_business_hours`	1,78 %

Threat intelligence (1)

Appel externe

`ip_abuse_score`	**8,32 %**

Réputation de l'IP via AbuseIPDB. Vaut 0 pour une IP privée (cache, timeout 2 s).

Les features comportementales n'existent pas dans le JSON brut : elles viennent d'un **historique en mémoire** par IP source, thread-safe, mis à jour **après** chaque calcul.

Labellisation comportementale

Niveau Wazuh	Label par défaut	Escalade si...
7 et plus	HIGH	toujours HIGH
5 – 6	MEDIUM	fréquence ≥ 10, ou répétition ≥ 8, ou abuse ≥ 80, alors HIGH
3 – 4	LOW	fréquence ≥ 15, ou répétition ≥ 10, ou abuse ≥ 80, alors MEDIUM

Fonction `make_label` de `train.py` , code complet en annexe.

Ce que ça permet

Deux alertes de même `rule.level` reçoivent des labels différents selon leur fréquence, leur répétition et la réputation de l'IP.

Point méthodologique

Le label est construit avec des critères liés aux features. Cela explique les scores très élevés, et nous y reviendrons dans les limites.

Random Forest et API Flask

POURQUOI RANDOM FOREST

Classes déséquilibrées

`class_weight='balanced'`, 200 arbres

Interprétable

`feature_importances_` pour l'analyse après coup

Portable

Fonctionne sur CPU et GPU, contrairement à XGBoost `device='cuda'`

SIX ENDPOINTS

HTTP	Endpoint	Rôle
POST	`/predict`	Classe l'alerte et lance la réponse
POST	`/respond/high`	Blocage IP, Telegram, e-mail
POST	`/respond/medium`	Telegram compact, e-mail
POST	`/respond/low`	Journal `incidents.json`
GET	`/health`	Disponibilité
GET	`/stats`	Statistiques

Shuffle n'a **qu'un seul bloc HTTP** à appeler : `/predict` déclenche lui-même le bon `/respond/<label>`.

Workflow Shuffle et actions selon le label

Webhook alerte Wazuh → **HTTP** POST /predict → **Shuffle Tools** extrait le label → **3 branches** avec une condition sur chaque flèche : HIGH, MEDIUM, LOW

HIGH

Blocage de l'IP par Wazuh Active Response (`firewall-drop`)

Telegram enrichi et e-mail structuré, avec le statut du blocage

MEDIUM

Notification de surveillance

Telegram compact et e-mail à l'analyste, sans le surcharger

LOW

Journalisation silencieuse

Une ligne dans `incidents.json` : règle, horodatage, agent

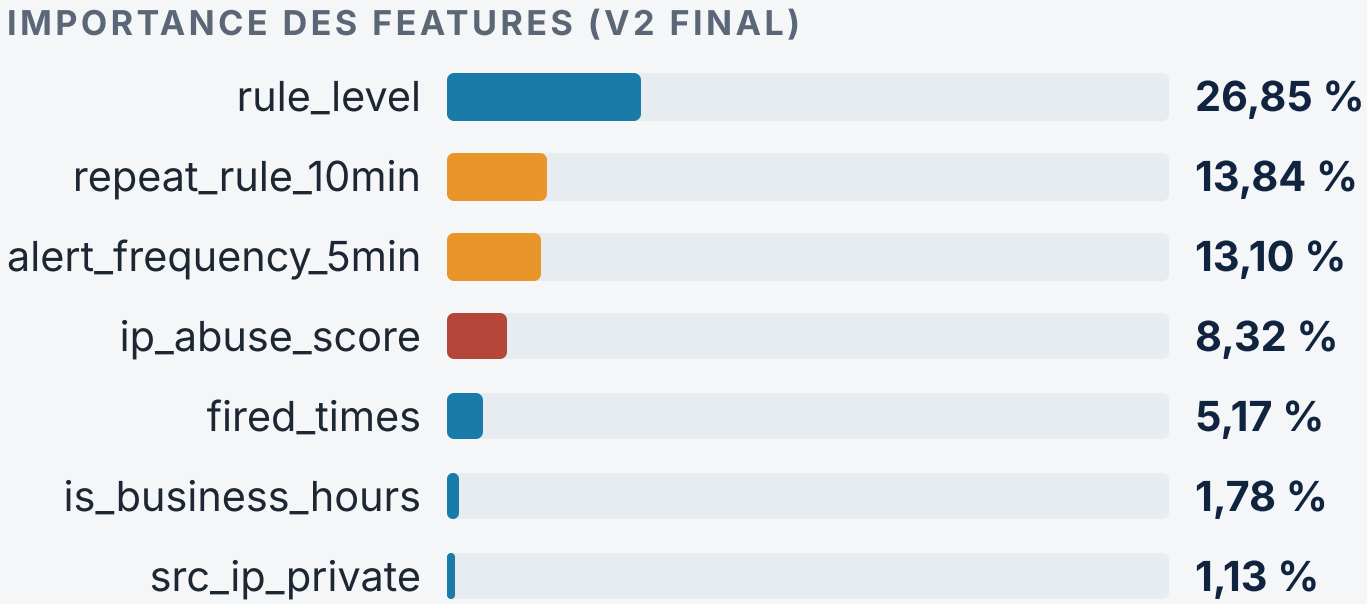
Le workflow tourne en permanence (Runtime Location : Running). L'analyste reçoit l'alerte avec l'action déjà exécutée.

Résultats du modèle V2

100 %

accuracy sur le jeu de test (789 / 789 alertes)

Classe	F1	Support
HIGH	1,00	352
LOW	1,00	302
MEDIUM	1,00	135



Barres orange : features comportementales, rouge : threat intelligence. Échelle : 0 à 100 %.

`rule_level` passe de **39,74 % (V1) à 26,85 % (V2)** : le comportement et la réputation d'IP pèsent désormais **environ 35 %** dans la décision.

04 · RÉSULTAT CLÉ

Escalade comportementale : la preuve

Même alerte, `rule.level = 5`, répétée 9 fois depuis la même source. Les 9 colonnes sont les 9 alertes successives.

	1	2	3	4	5	6	7	8	9
Répétitions sur 10 min repeat_rule_10min	0	1	2	3	4	5	6	7	**8**
Seuil Shuffle rule.level ≥ 7	MEDIUM	MEDIUM	MEDIUM	MEDIUM	MEDIUM	MEDIUM	MEDIUM	MEDIUM	MEDIUM
Notre Random Forest	MEDIUM	MEDIUM	MEDIUM	MEDIUM	MEDIUM	MEDIUM	MEDIUM	MEDIUM	HIGH

À la 9e alerte, `repeat_rule_10min` atteint 8 : la classe bascule en **HIGH (58 %)**. Les alertes 1 à 8 sont MEDIUM, entre 49 et 53 %. Un seuil sur `rule.level` ne peut pas faire cela.

Deuxième preuve : une IP publique avec un score AbuseIPDB de 100 est classée HIGH, même avec un niveau Wazuh faible.

Tests du pipeline de bout en bout

LOW

98 %

Règle 5715, authentification SSH réussie

Journalisation dans `incidents.json`

MEDIUM

63 %

Règle 31103, attaque XSS (niveau 6)

Telegram compact et e-mail analyste

HIGH

94 %

Règle 5712, brute force SSH (niveau 10)

Blocage IP, Telegram enrichi, e-mail

Trois alertes successives traitées en temps réel, sans configuration entre chaque. Score = confiance du modèle.

Réponse HIGH vérifiée

IP `185.220.101.1` (AbuseIPDB 100). L'API renvoie `blocked: true`, `telegram_sent: true`, `email_sent: true`, exécutés en parallèle.

1,231 s

temps de réponse mesuré (objectif BNF1 : < 2 s)

Mes contributions

Problème et idée

Constat que le SOAR sans triage sature comme le SIEM. Proposition de la couche ML, validée par M. Erraji.

Diagnostic CICIDS

Analyse des features dominantes, puis changement d'approche vers un dataset natif.

Feature engineering

Features temporelles et threat intelligence, historique en mémoire thread-safe, intégration AbuseIPDB avec cache.

Labellisation dynamique

Schéma `make_label` avec fréquence, répétition et score de réputation.

API Flask V2

6 endpoints, appel automatique de `/respond/*`, blocage via l'API JWT de Wazuh.

Infrastructure et tests

Docker, LVM, Orborus, ossec.conf. Trois scénarios complets et la démonstration d'escalade.

Difficultés rencontrées et solutions

Problème	Solution adoptée
Modèle CICIDS prédit toujours BENIGN	Dataset natif Wazuh et features sémantiques
XGBoost `cuda` non portable entre GPU et CPU	Random Forest entraîné sur la VM, en CPU
Orborus : timeout des workers Shuffle	`SWARM=off` et `BASE_URL=192.168.56.100`
`ossec.conf` écrasé à chaque redémarrage	Modifier `wazuh_manager.conf` (bind mount)
`/home` plein à 100 % (deux fois)	Extension du volume LVM via VBoxManage
Docker daemon : « too many open files »	Réglage `sysctl` inotify et redémarrage de containerd

Que valent vraiment les 100 % ?

Cohérence interne, pas généralisation

Les labels sont construits à partir de critères corrélés aux features : `rule_level` sert dans `make_label`, et `rule_id_enc` l'encode indirectement. Le modèle atteint 100 % même sans `rule_level`.

Ce qui le distingue d'un seuil

Il décide dans un espace à **11 dimensions** : un score de confiance gradué (49 % à 88 %) et une escalade comportementale, impossibles avec `rule.level ≥ 7`.

LIMITES ASSUMÉES

- Dataset de laboratoire : 3 945 alertes, IP simulées
- `ip_abuse_score` vaut 0 sur les IP privées
- `is_business_hours` (1,78 %) : timestamps peu variés
- Pas d'étude d'ablation formelle
- Blocage validé côté API ; un agent Wazuh est requis sur la cible
- API en mode développement (Gunicorn en production)

Perspectives

1 Dataset de production

Collecter des alertes OCP réelles pour gagner en diversité.

2 Agents Wazuh sur les serveurs

Activer le blocage IP effectif sur l'infrastructure OCP.

3 Conteneuriser l'API

Docker et Gunicorn dans le docker-compose global.

4 Détection multi-étapes

LSTM sur des séquences d'alertes pour des attaques complexes.

5 Mesures de latence robustes

Plus de 50 mesures pour une moyenne et un écart-type.

CONCLUSION

Ce qu’il faut retenir

Le problème

Un SOAR sans triage reproduit la saturation du SIEM : 100 alertes par seconde à l'OCP.

La solution

Un Random Forest entre Wazuh et Shuffle, entraîné sur des alertes natives, avec 11 features comportementales et de réputation d'IP.

Le résultat

Pipeline complet en 1,231 s : blocage IP, Telegram, e-mail pour HIGH ; notification pour MEDIUM ; journal pour LOW.

Apport central : deux alertes de même niveau Wazuh obtiennent des criticités différentes selon leur comportement. Une alerte de niveau 5 devient HIGH après 8 répétitions.

Une base technique pour intégrer l'IA au futur déploiement SOAR de l'OCP, avec des limites connues et des pistes claires pour les lever.



Merci de votre attention

Place à vos questions

ANNEXE A

Environnement technique

Composant	Spécification
Hyperviseur	Oracle VirtualBox
Système	Ubuntu Server 24.04 LTS
Ressources	10 Go de RAM, 4 vCPU, disque de 100 Go (LVM)
Réseau	Host-Only 192.168.56.100/24 et NAT
Accès aux interfaces	Tunnel SSH `-L 3001:localhost:3001`
Python	3.12.3 (venv), scikit-learn 1.4.2, Flask
Conteneurs	7 : Wazuh (Manager, Indexer, Dashboard) et Shuffle (Frontend, Backend, OpenSearch, Orborus)

ANNEXE B

Chiffres du dashboard Wazuh

5 342
alertes sur la période de test

839
échecs d'authentification

287
authentifications réussies

11
alertes de niveau 12 ou plus

Trois agents actifs : **Sejjil-1**, **wazuh.manager** et **windows-agent**. Les techniques MITRE ATT&CK dominantes sont *Password Guessing* (T1110) et *SSH* (T1021.004), cohérentes avec les scénarios de brute force simulés.

Le dataset d'entraînement (3 945 alertes) exclut les 1 397 alertes système (démarrages, SCA, rootcheck, netstat).

ANNEXE C

Code de la labellisation : `make_label`

```
def make_label(row):
    level, freq = row['rule_level'], row['alert_frequency_5min']
    repeat, abuse = row['repeat_rule_10min'], row['ip_abuse_score']
    if level >= 7:
        return 'HIGH'
    if level >= 5:
        if freq >= 10 or repeat >= 8 or abuse >= 80:
            return 'HIGH' # escalade comportementale
        return 'MEDIUM'
    if level >= 3:
        if freq >= 15 or repeat >= 10 or abuse >= 80:
            return 'MEDIUM' # escalade comportementale
        return 'LOW'
    return 'LOW'
```